Solution: Longest Valid Parentheses

Problem Statement

You are given a string containing just the characters '(' and ')'. Your task is to find the length of the longest valid (well-formed) parentheses substring.

Example 1:

- **Input**: `"(()"`
- **Expected Output**: `4`
- **Justification**: The entire string is a valid parentheses substring.

Example 2:

- **Input**: `")()())"`
- **Expected Output**: `4`
- **Justification**: The longest valid parentheses substring is `"()()"`.

Example 3:

- **Input**: `"(()"`
- **Expected Output**: `2`
- **Justification**: The longest valid parentheses substring is `"()"`.

Solution

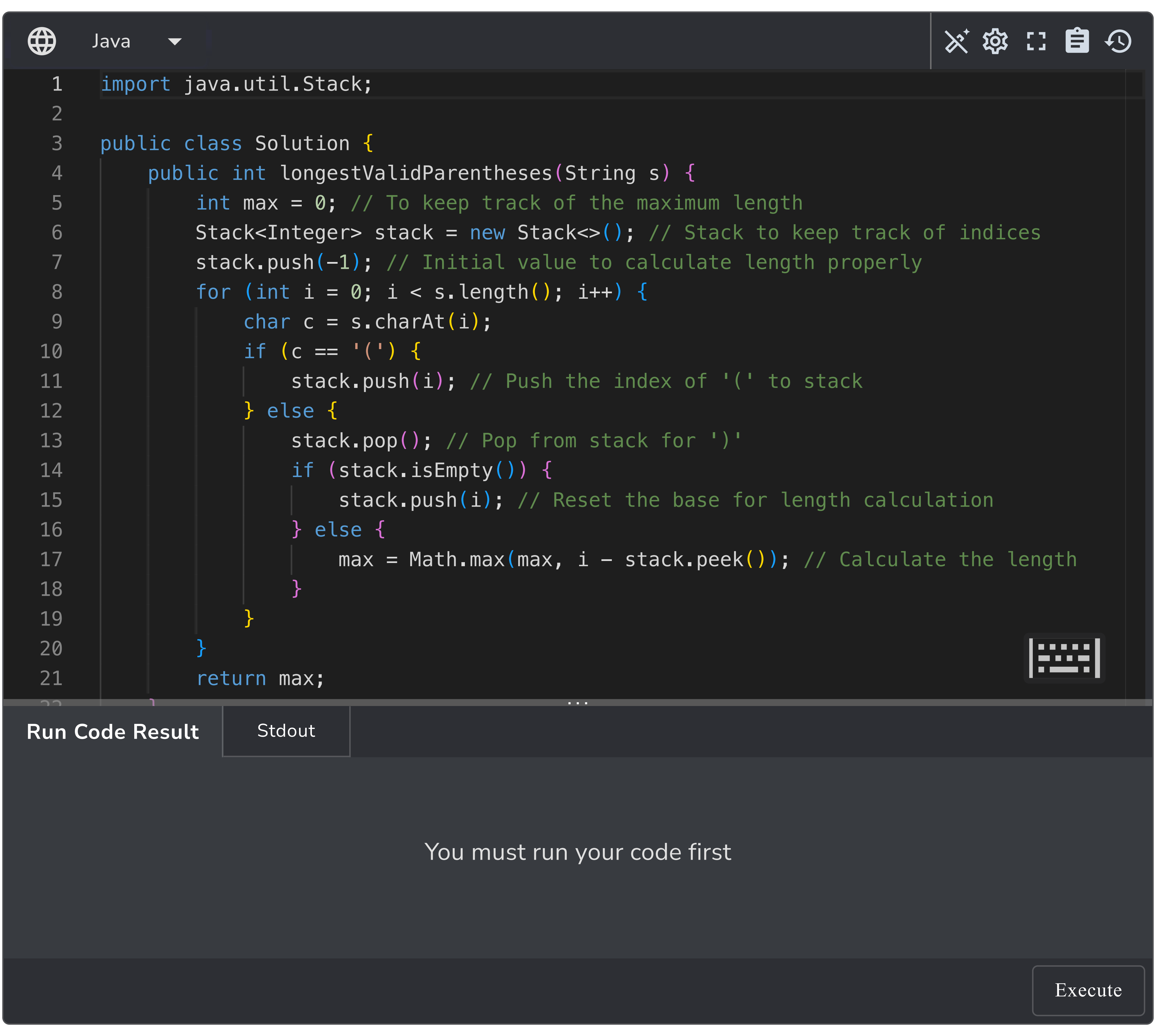
- **Understanding the Problem**:
  - The problem involves finding the longest sequence of valid parentheses.
  - A stack data structure can be used to keep track of the indices of the invalid parentheses.
- **Approach**:
  - Initialize a stack and push `-1` onto it as a marker for the base.
  - Iterate through the string and for each character:
    - If it is `'('`, push its index onto the stack.
    - If it is `')'`, pop an element from the stack.
      - If the stack is not empty, calculate the length of the valid parentheses substring by subtracting the current index with the top of the stack.
      - If the stack is empty, push the current index onto the stack to serve as the new base marker.
- **Why This Approach Will Work**:
  - The stack keeps track of the indices of invalid parentheses, allowing us to easily calculate the length of valid substrings.
  - By continuously calculating the length and updating the maximum length, we ensure finding the longest valid substring.

Algorithm Walkthrough

Consider the input `")()())"`:

- Initialize stack with `-1`: `stack = [-1]`
- Iterate through the string:
  - At index `0`: `')'`
    - Pop from stack: `stack = []`
    - Stack is empty, push index `0` onto stack: `stack = [0]`
  - At index `1`: `'('`
    - Push index `1` onto stack: `stack = [0, 1]`
  - At index `2`: `')'`
    - Pop from stack: `stack = [0]`
    - Calculate length: `2 - 0 = 2`
  - At index `3`: `'('`
    - Push index `3` onto stack: `stack = [0, 3]`
  - At index `4`: `')'`
    - Pop from stack: `stack = [0]`
    - Calculate length: `4 - 0 = 4`
  - At index `5`: `')'`
    - Pop from stack: `stack = []`
    - Stack is empty, push index `5` onto stack: `stack = [5]`
- The longest valid parentheses substring is of length `4`.

Code



```java
import java.util.Stack;

public class Solution {
    public int longestValidParentheses(String s) {
        int max = 0; // To keep track of the maximum length
        Stack<Integer> stack = new Stack<>(); // Stack to keep track of indices
        stack.push(-1); // Initial value to calculate length properly
        for (int i = 0; i < s.length(); i++) {
            char c = s.charAt(i);
            if (c == '(') {
                stack.push(i); // Push the index of '(' to stack
            } else {
                stack.pop(); // Pop from stack for ')'
                if (stack.isEmpty()) {
                    stack.push(i); // Reset the base for length calculation
                } else {
                    max = Math.max(max, i - stack.peek()); // Calculate the length
                }
            }
        }
        return max;
```

You must run your code first

Complexity Analysis

- **Time Complexity**: O(n), where n is the length of the input string. The algorithm traverses the string once.
- **Space Complexity**: O(n), where n is the length of the input string. In the worst case, the stack will store all characters of the string.

← Previous

Longest Valid Parentheses
(Hard)

☐ Mark as Completed